Java AML 反洗钱系统项目说明书

Java AML 反洗钱合规管理平台

项目教学演示文档

2026 年 10 月

目录

第一章 项目概述 **2**

1.1 背景与用途 2

1.2 实际技术栈 2

1.3 真实功能范围 2

第二章 系统架构 **4**

2.1 总体结构 4

2.2 核心业务关系 4

第三章 主要功能 **5**

3.1 登录与权限 5

3.2 客户、账户与交易 5

3.3 交易监控规则与预警 7

3.4 案件调查 9

3.5 名单筛查与风险评分 10

3.6 仪表盘、报表和审计 12

第四章 数据库设计 **14**

4.1 迁移和约束 14

第五章 部署与验收 **15**

5.1 本地运行 15

5.2 真实测试结果 15

第六章 结论与边界 **16**

第一章　项目概述

1.1　背景与用途

本项目是面向课程实验、教师验收和源码学习的反洗钱合规管理平台。系统围绕客户、账户和交易建立可追溯的业务链路，通过规则识别异常交易并生成预警，随后支持案件调查、名单筛查、风险评分、统计报表和审计查询。

系统中的阈值、规则和评分模型均为教学示例，不构成法律、监管或真实授信结论。运行截图和端到端测试只使用合成或模拟数据。

1.2　实际技术栈

层次	实际采用技术
后端	Java 21、Spring Boot 3.5.7、Spring Security、JWT、JPA/Hibernate、JdbcTemplate
数据库	PostgreSQL 16.10、Flyway 11.7.2，迁移版本 V1–V6
规则与接口	Drools/KIE 10.1.0、Jakarta Validation、springdoc-openapi 2.8.13
前端	Vue 3.5、TypeScript 5.9、Vite 7.3、Element Plus、Pinia、Axios、ECharts
部署与测试	Docker Compose、Nginx、JUnit 5、Mockito、MockMvc、H2 PostgreSQL 模式与真实 PostgreSQL E2E

1.3　真实功能范围

已实现登录鉴权、客户、账户、交易查询和 CSV 导入、R001–R003 三类监控规则、预警管理、案件调查、Mock 名单筛查及人工复核、客户/交易风险评分、数据库实时仪表盘、CSV 导出和审计查询。主链路“客户 → 账户 → 交易 → 规则 → 预警 → 案件 → 关闭”已经真实 HTTP 和 PostgreSQL 验证。

尚未实现完整 KYC 编辑工作流、用户角色后台、R004–R008 算法、复杂图/循环转账规则、正式监管报送生命周期。Yente 和 Fineract 只有适配器、契约测试与失败处理，没有真实目标系统联调。

第二章　系统架构

2.1　总体结构

系统为前后端分离的模块化单体。浏览器访问 Vue 单页应用，Nginx 代理后端 API；Spring Boot 完成认证、业务和数据访问；PostgreSQL 保存业务历史与审计数据。Yente、Fineract 位于可选集成边界，默认关闭，外部不可用时本地功能仍可运行。

后端源代码根目录为 `backend/src/main/java/com/example/aml`，按照 auth、security、customer、account、transaction、rule、alert、casework、screening、risk、dashboard、audit 和 integration 等领域组织。

2.2　核心业务关系

客户可以关联账户，账户承载交易；规则引擎按时间窗口查询交易并产生规则命中，命中生成预警；预警可以升级为案件，案件记录分派、笔记、证据、状态历史和关闭结论。筛查、风险评分和审计记录从侧面补充业务证据。

第三章　主要功能

3.1　登录与权限

登录接口校验 BCrypt 密码并签发 JWT，后端使用 Spring Security 和方法权限控制接口。前端菜单按权限过滤，但权限最终由后端执行。已知缺陷是失败登录计数和失败审计会随事务回滚，不能将账户锁定描述为验证通过。

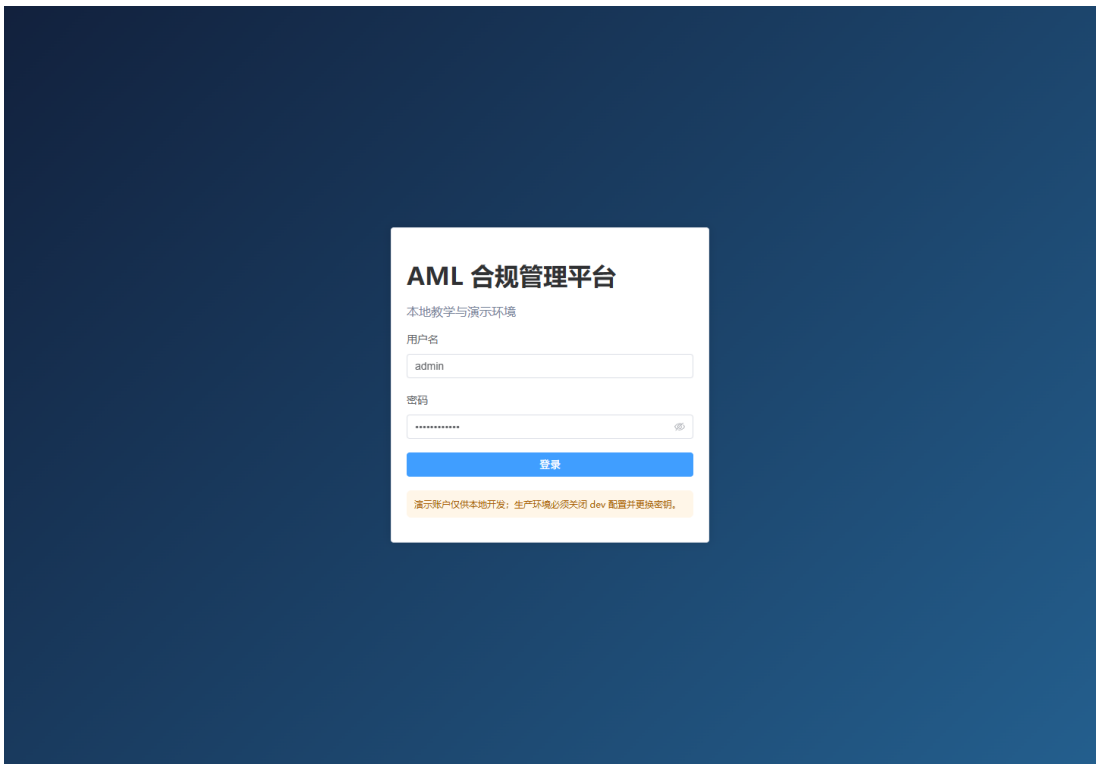


图 3.1: 系统登录页，密码保持遮蔽（模拟/合成数据）

3.2　客户、账户与交易

客户模块支持分页查询、新增、编辑和停用；账户模块支持客户关联、查询和状态管理；交易模块支持查询、平台 CSV 与 AMLSim 默认列 CSV 导入、字段校验和来源外部标识去重。Fineract 适配仅覆盖储蓄账户流水读取，当前未配置真实服务。

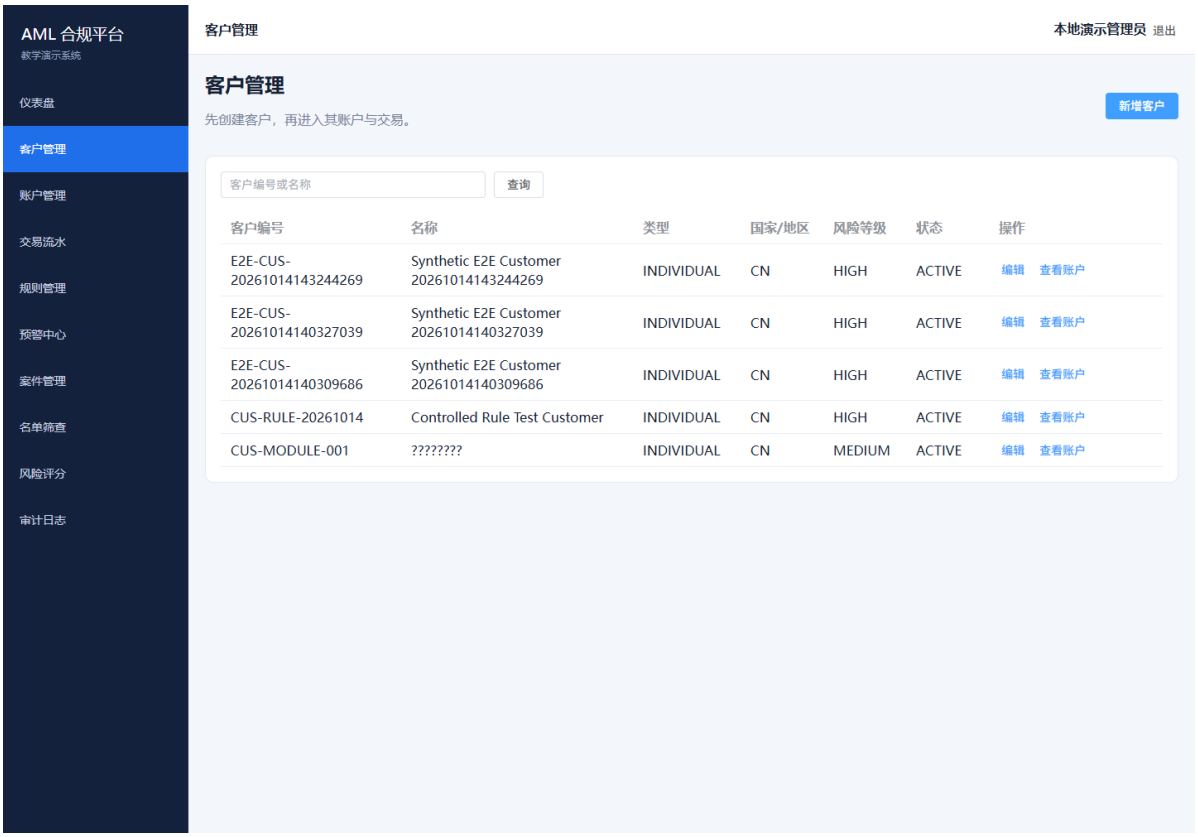


图 3.2: 客户管理页（模拟/合成数据）

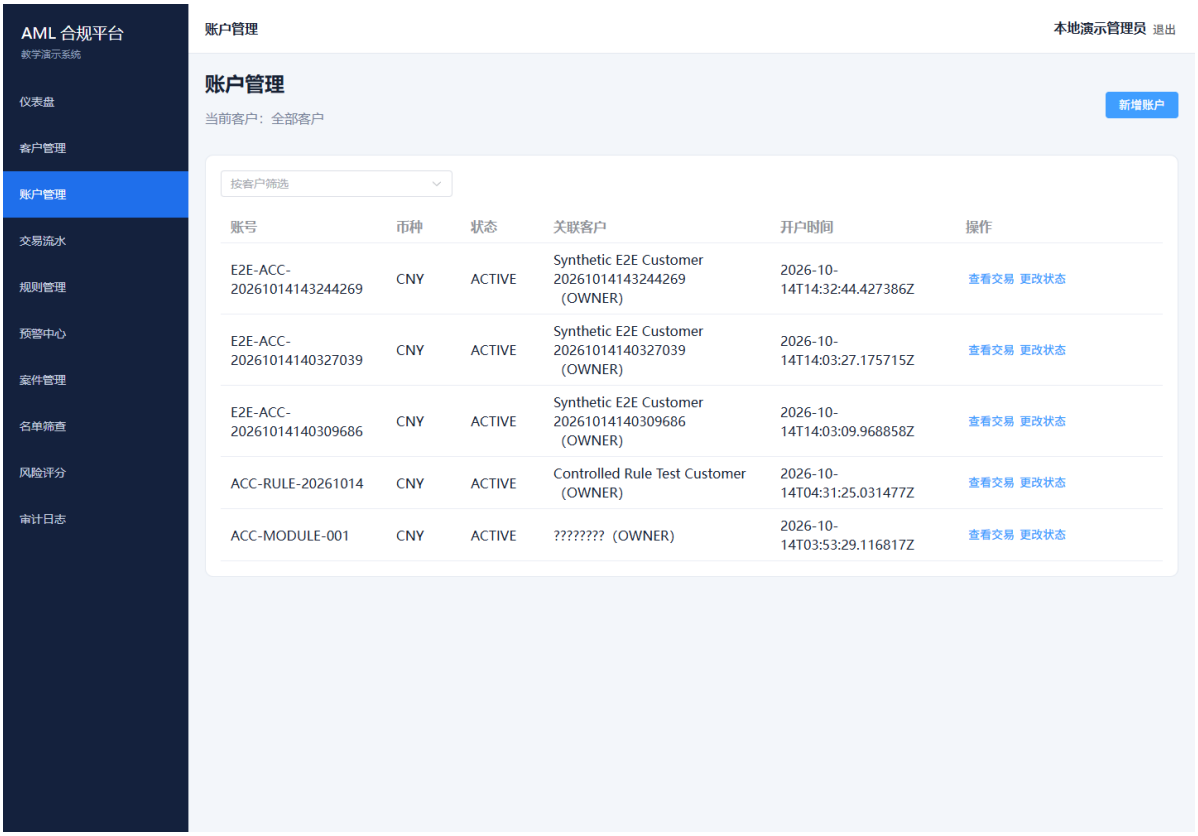


图 3.3: 账户管理页（模拟/合成数据）

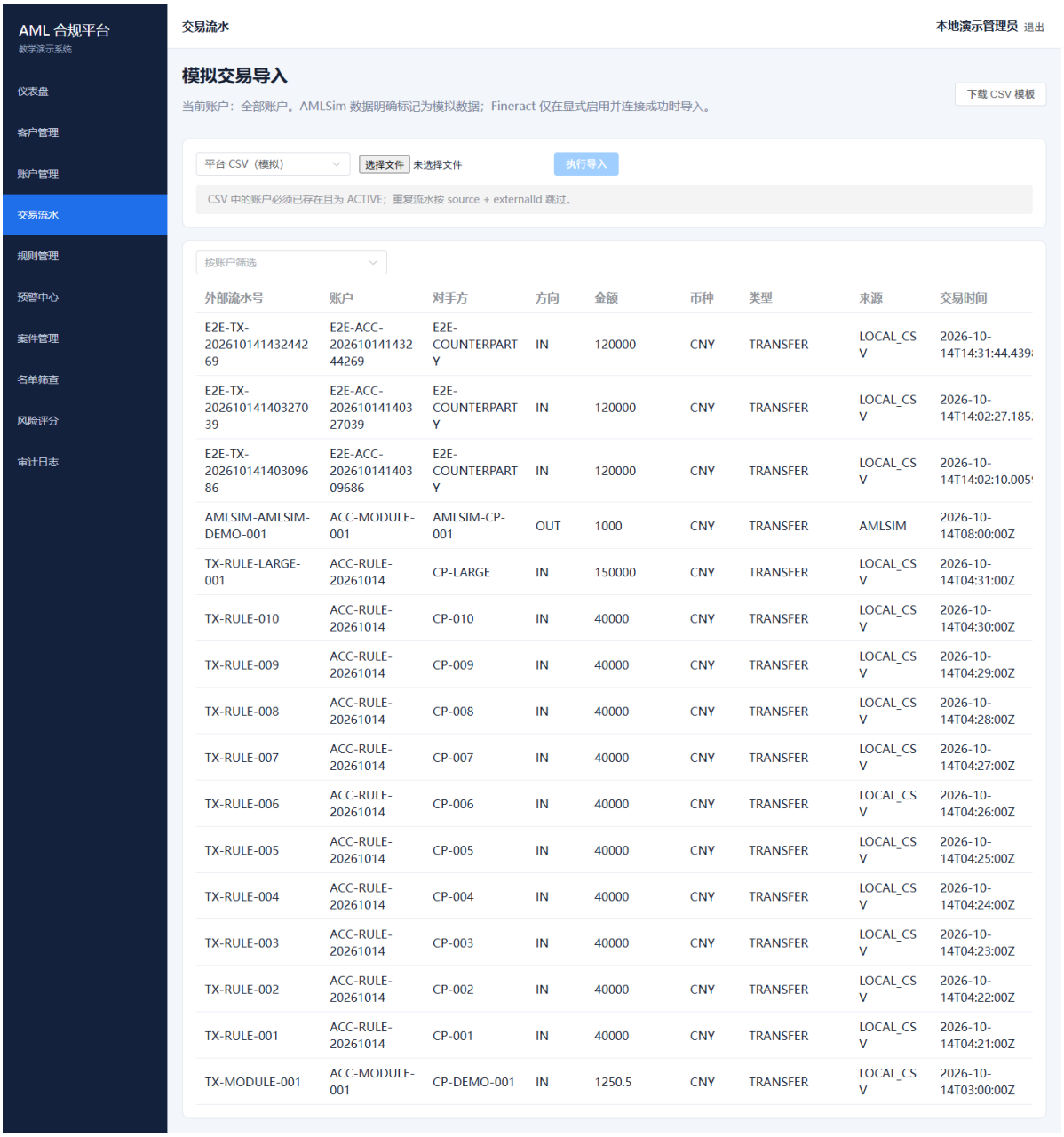


图 3.4: 交易查询与导入页（模拟/合成数据）

3.3 交易监控规则与预警

数据库保存规则元数据、版本、执行记录和命中记录。Drools 10.1.0 执行 R001 单笔大额、R002 短时高频和 R003 窗口累计交易。确定性命中标识与数据库约束共同避免同一窗口重复生成命中和预警。R004–R008 仅为禁用的规划记录。



图 3.5: 交易监控规则页（模拟/合成数据）

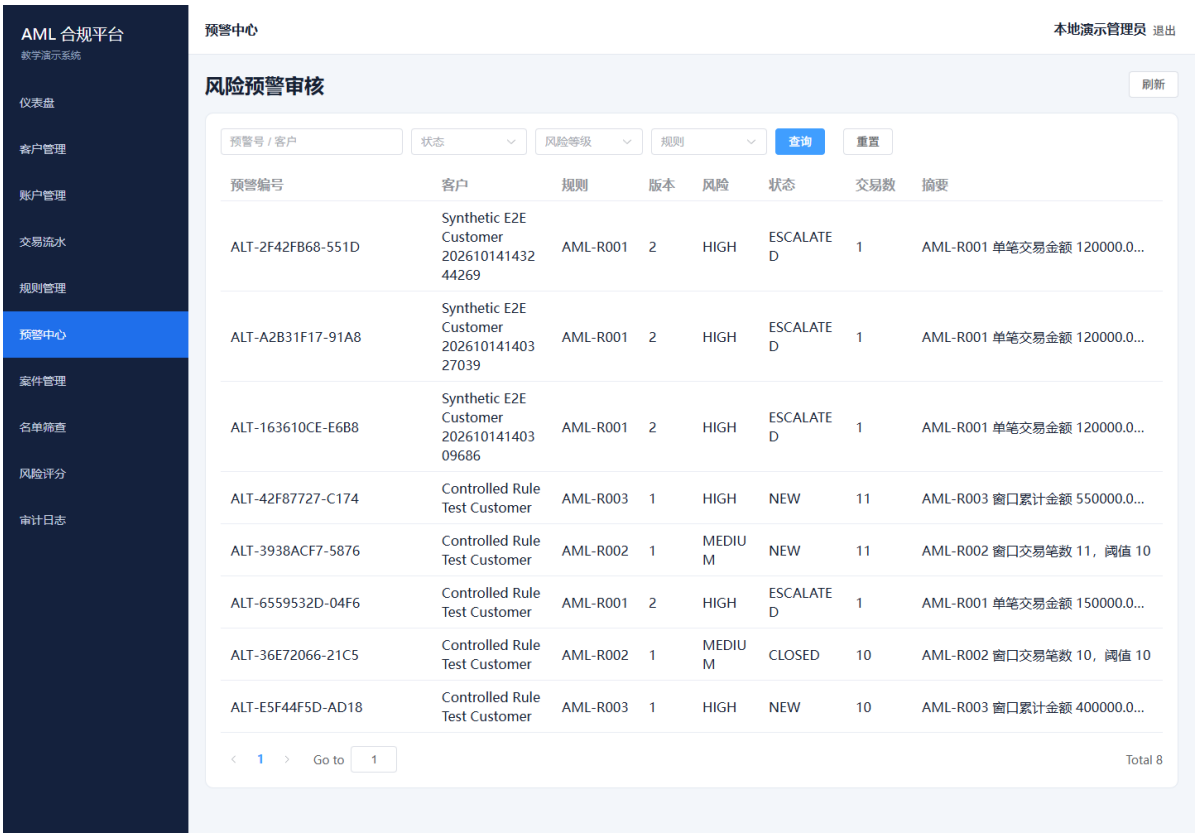


图 3.6: 预警查询与处置页（模拟/合成数据）

3.4 案件调查

预警可升级为案件。案件支持调查员分派、调查笔记、证据上传/下载/逻辑移除、SHA-256、状态历史和关闭结论。状态机限制为 OPEN→INVESTIGATING→PENDING_REVIEW→CLOSED，并允许复核退回。证据保存在本地 Docker 卷，未实现病毒扫描和对象存储。

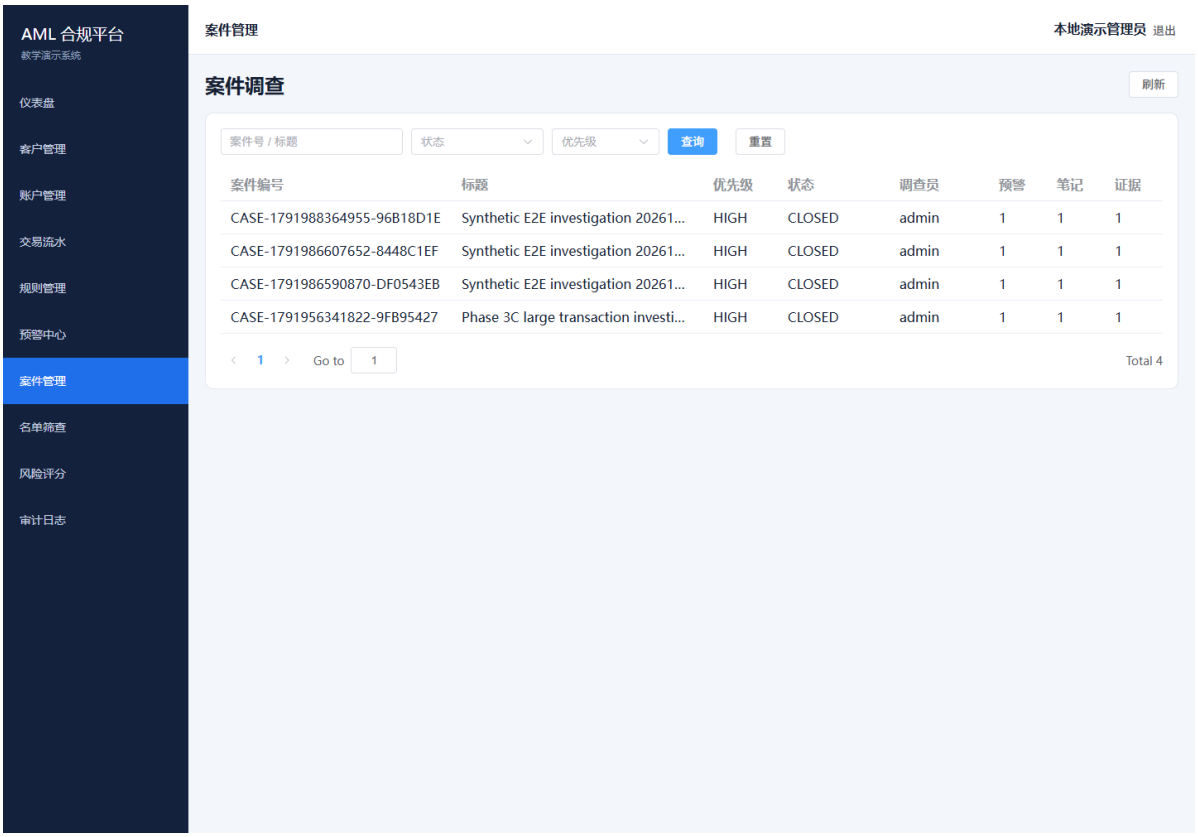


图 3.7: 案件调查工作台（模拟/合成数据）

3.5 名单筛查与风险评分

ScreeningProvider 抽象提供 Mock 和 Yente 两种适配器。Mock 结果明确标记为 SIMULATED；Yente 支持超时、非 2xx 和畸形响应处理，真实服务失败不会被伪装为“未命中”。风险评分保留当前快照和只追加历史，属于确定性的教学模型。

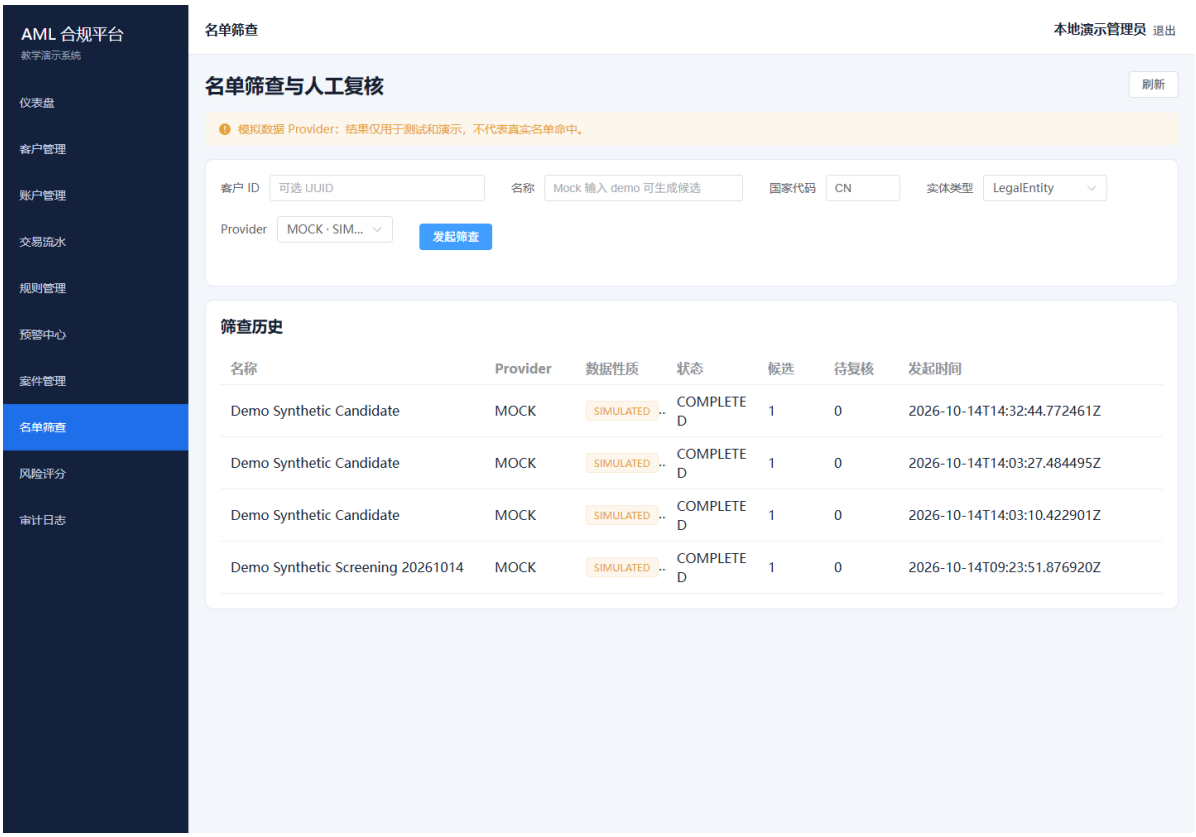


图 3.8: 名单筛查与人工复核页（模拟/合成数据）

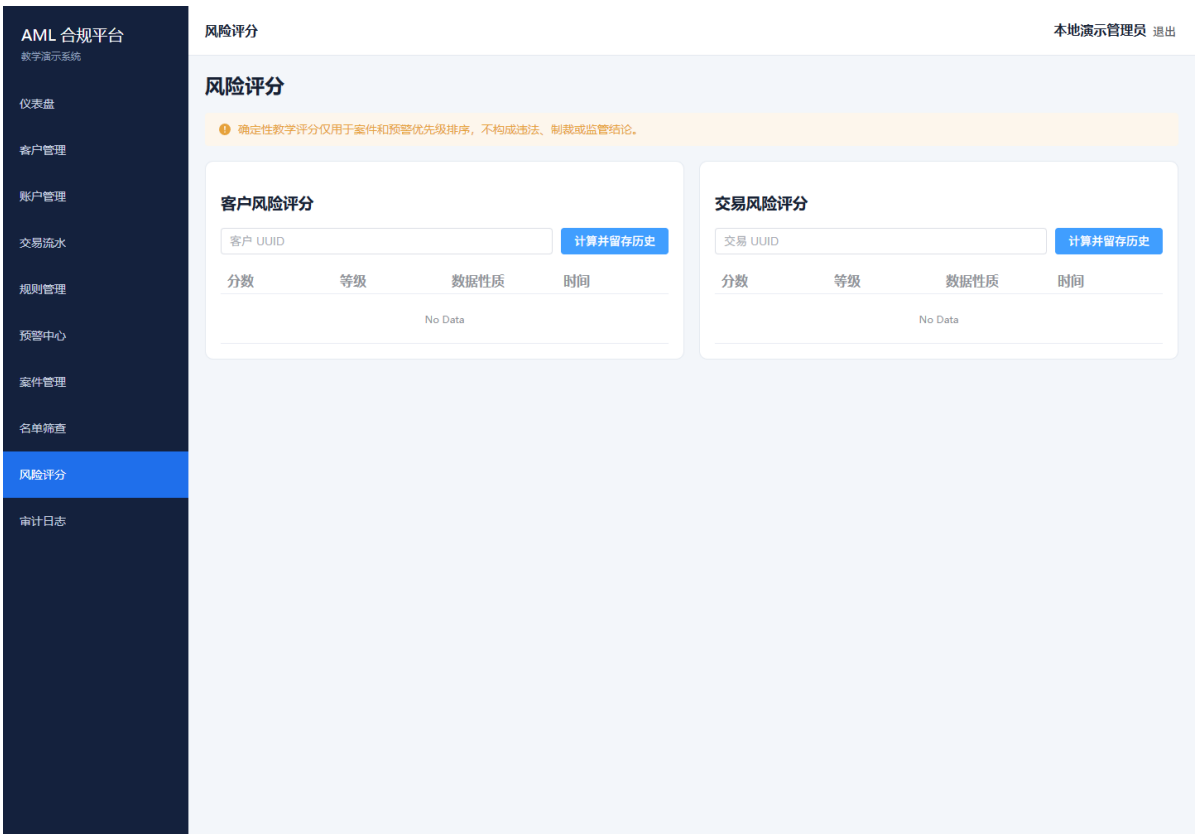


图 3.9: 客户与交易风险评分页（模拟/合成数据）

3.6 仪表盘、报表和审计

仪表盘通过后端实时查询 PostgreSQL，显示客户风险分布、预警趋势、规则命中、案件状态和优先级统计，不使用前端写死值。系统支持多类 UTF-8 CSV 即时导出。审计页支持筛选、分页和导出；登录失败审计缺陷必须同时披露。

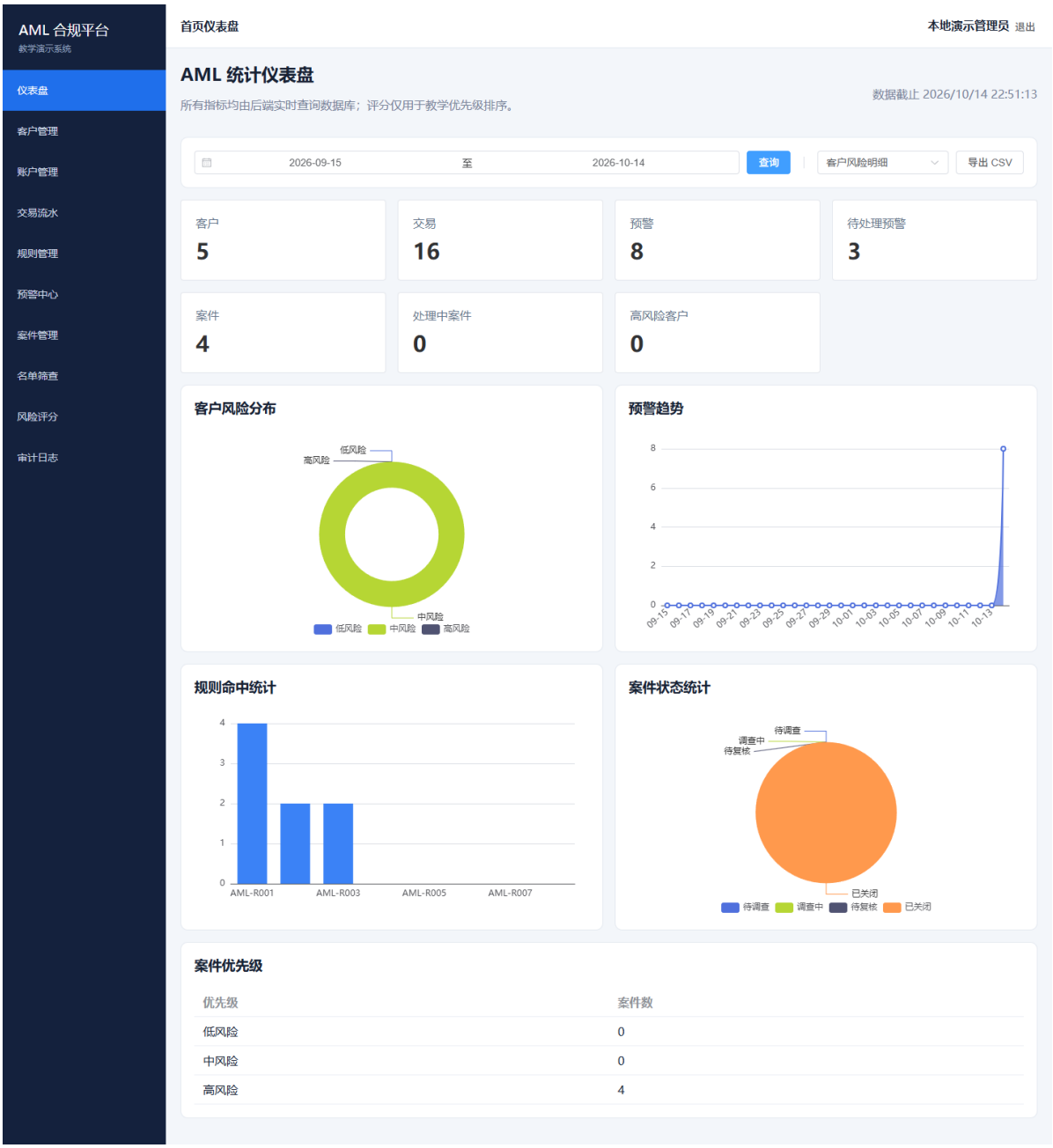


图 3.10: AML 统计仪表盘（模拟/合成数据）

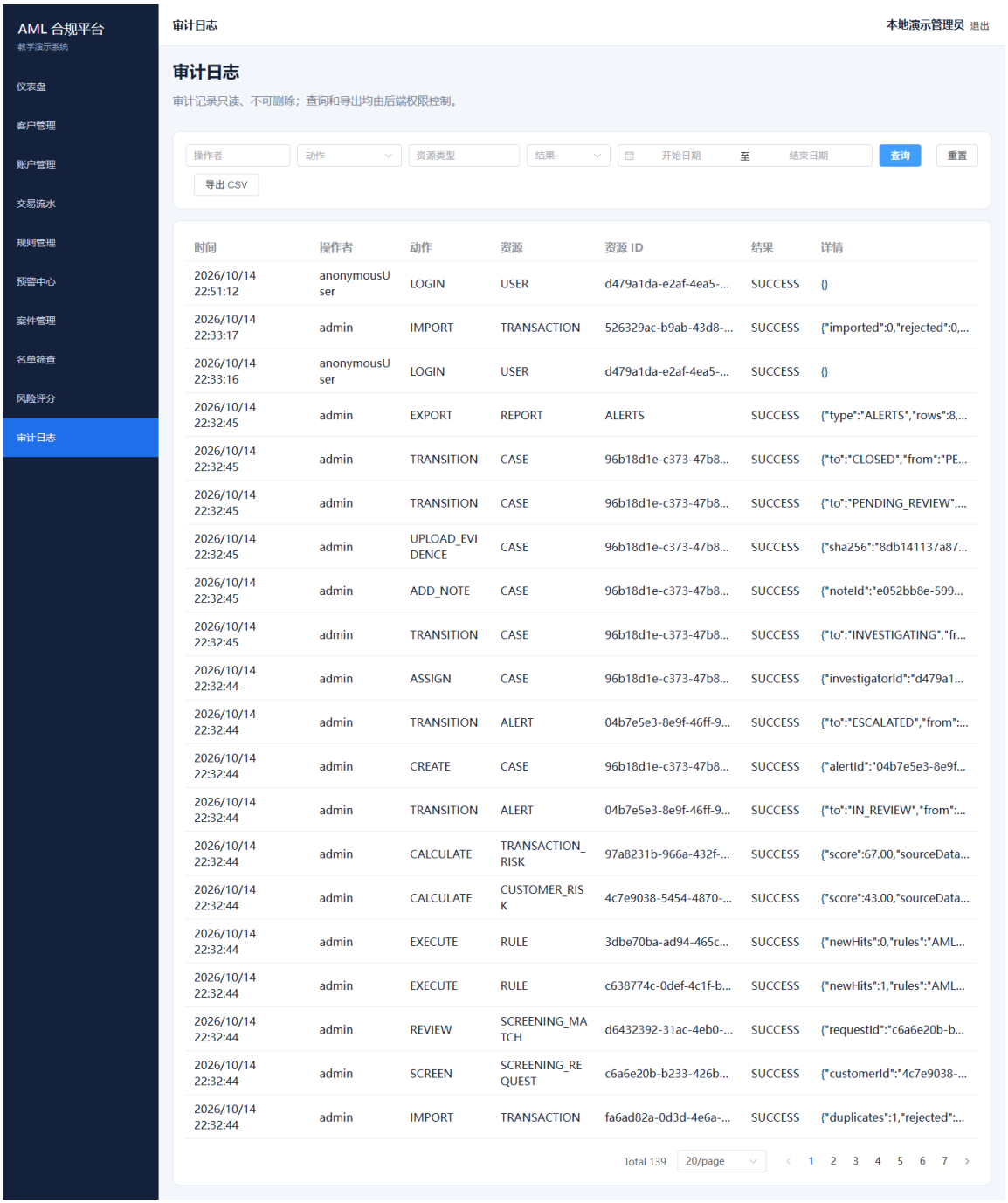


图 3.11: 审计查询页（模拟/合成数据）

第四章　数据库设计

4.1　迁移和约束

数据库结构仅通过 Flyway V1–V6 演进，Hibernate 使用 schema validation。独立空库验证得到 34 张应用表、49 个外键和 81 个索引，另有 Flyway 历史表。主键通常为 UUID，金额使用 numeric(19,4)，时间使用 timestamptz。

领域	主要表
RBAC	sys_user、sys_role、sys_permission 及关联表
客户账户	aml_customer、aml_account、aml_account_customer
交易	aml_transaction、aml_transaction_import_batch
规则预警	aml_rule*、aml_rule_hit*、aml_alert*
案件	aml_case、aml_case_alert、aml_case_note、aml_case_evidence、aml_case_history
筛查评分	aml_screening*、aml_customer_risk*、aml_transaction_risk*
审计	sys_audit_log

第五章　部署与验收

5.1　本地运行

主机需具备 Docker Desktop；也可分别安装 JDK 21、Maven、Node.js 和 PostgreSQL。复制 `.env.example` 为未跟踪的 `.env` 并替换数据库密码、JWT 密钥和演示密码。完整启动命令为：

```
docker compose --profile app up -d --build
```

前端地址为 http://localhost:5173，OpenAPI 为 http://localhost:8080/swagger-ui.html，健康检查为 http://localhost:8080/actuator/health。

5.2　真实测试结果

项目	实际结果	状态
后端测试	JDK 21 执行 Maven clean verify，32 tests，0 failures/errors/skipped	通过
前端构建	vue-tsc 与 Vite 成功，2300 modules	通过
数据库迁移	独立 PostgreSQL 执行 V1–V6，schema validation 成功	通过
Compose/HTTP	三容器运行，健康、OpenAPI 和前端均正常	通过
核心 E2E	客户到账户、交易、规则、预警、案件关闭、评分、统计、审计和 CSV	通过
真实外部系统	Yente、Fineract 未配置；只通过适配器契约测试	未实连

第六章　结论与边界

本项目形成了可运行、可追溯的教学 AML 主链路，适合展示 Spring Boot、Vue 和 PostgreSQL 的业务建模、规则监控与审计设计。完整证据应从 `docs/project-review.md`、`docs/audit/`、数据库迁移、测试源码和 E2E 脚本交叉核对。

归档前仍需由所有者决定项目许可证，确认 GitHub 账号、仓库名和可见性。生产部署必须禁用演示默认凭据、使用强密钥并补齐 TLS、备份、病毒扫描和可观测性。